

Estructura de ramas Git en *Inventiory* basado en el **plan de desarrollo**. Se utilizará un esquema basado en GitFlow, adaptado al flujo de trabajo. Como *Inventiory* tiene funcionalidades bien definidas, se crearán ramas específicas para cada una.

✦ Estructura de Ramas para *Inventiory*

1- Rama Principal (main)

- Contiene el código en producción (lo que está funcionando y estable).
- Se actualiza solo con versiones estables.

2- Rama de Desarrollo (develop)

- Aquí se integran todas las funcionalidades antes de pasar a producción.
- Los colaboradores trabajarán en esta rama indirectamente, ya que crearán sub-ramas para cada funcionalidad.
- Se fusiona en main cuando hay una versión lista para producción.

3- Ramas de Funcionalidades (feature/nombre-de-la-funcionalidad)

- Se crean a partir de develop para desarrollar cada módulo de *Inventiory*.
 - Cada funcionalidad tiene su propia rama.
 - Se fusionan en develop mediante Pull Requests cuando están listas.
-

✦ Ramas basadas en el plan de desarrollo

Como *Inventiory* tiene funcionalidades bien definidas, usaremos las siguientes ramas:

◆ Gestión de Usuarios y Autenticación

- feature/autenticacion-jwt → Para implementar la autenticación con JWT.
- feature/gestion-usuarios → Para la administración de usuarios.

◆ Gestión de Clientes

- feature/gestion-clientes → Para registrar, ver y modificar clientes.

◆ Gestión de Productos

- feature/gestion-productos → Para registrar, listar y actualizar productos.

◆ **Control de Stock**

- feature/control-stock → Para registrar entradas y salidas de stock.

◆ **Ventas y Facturación**

- feature/registro-ventas → Para registrar las ventas.
- feature/facturacion → Para la generación de facturas electrónicas.

◆ **Informes y Análisis**

- feature/informes → Para la generación de informes y análisis de ventas y stock.

✦ **Flujo de Trabajo**

1- **Inicio de una funcionalidad**

- Cuando empieces una nueva funcionalidad, creas una rama desde develop:

```
git checkout develop
```

```
git checkout -b feature/gestion-productos
```

2- **Desarrollo y Commits**

- Trabajas en la funcionalidad y haces commits normalmente:

```
git add .
```

```
git commit -m "Agregada funcionalidad para registrar productos"
```

3- **Subir cambios a GitHub**

- Cuando termines la funcionalidad, subes la rama a GitHub:

```
git push origin feature/gestion-productos
```

4- **Pull Request y Revisión**

- Abres un **Pull Request (PR)** en GitHub para fusionar feature/gestion-productos en develop.
- Los colaboradores revisarán el código antes de aceptarlo.

5- **Fusión en develop y Eliminación de la Rama**

- Si todo está bien, se aprueba el PR y se fusiona en develop.
- Se borra la rama feature/gestion-productos.

✦ Resumen Visual de las Ramas

main



Guía Completa: Gestión de Ramas en Git para Inventory

1. Configuración Inicial

Se crea el repositorio en Github, y se agregan a los colaboradores necesarios. Luego, si aún no tienes el repositorio clonado en tu máquina, ejecúta:

```
git clone https://github.com/MichaelIllescas/Inventory.git
```

```
cd Inventory
```

Verifica en qué rama estás:

```
git branch
```

Si solo tienes main, necesitas crear develop.

2. Crear la Rama develop

Esta rama será la base para todo el desarrollo.

```
git checkout -b develop
```

```
git push origin develop
```

✅ Ahora tienes develop en GitHub y listo para comenzar a trabajar.

3. Crear y Trabajar en Funcionalidades

Cada funcionalidad tendrá su propia **rama feature/** y se creará a partir de develop.

3.1 Autenticación con JWT

```
git checkout develop
```

```
git checkout -b feature/autenticacion-jwt
```

```
git push origin feature/autenticacion-jwt
```

Subir cambios

```
git add .
```

```
git commit -m "Implementada autenticación con JWT"
```

```
git push origin feature/autenticacion-jwt
```

Cuando termines, crea un **Pull Request** en GitHub para fusionarlo con develop.

Fusionar en develop y eliminar rama:

```
git branch -d feature/autenticacion-jwt
```

```
git push origin --delete feature/autenticacion-jwt
```

3.2 Otras funcionalidades (Mismo proceso para cada una)

Gestión de Usuarios

```
git checkout develop
```

```
git checkout -b feature/gestion-usuarios
```

git push origin feature/gestion-usuarios

Gestión de Clientes

git checkout develop

git checkout -b feature/gestion-clientes

git push origin feature/gestion-clientes

Gestión de Productos

git checkout develop

git checkout -b feature/gestion-productos

git push origin feature/gestion-productos

Control de Stock

git checkout develop

git checkout -b feature/control-stock

git push origin feature/control-stock

Ventas y Facturación

git checkout develop

git checkout -b feature/registro-ventas

git push origin feature/registro-ventas

git checkout develop

git checkout -b feature/facturacion

git push origin feature/facturacion

Informes y Análisis

git checkout develop

git checkout -b feature/informes

git push origin feature/informes

Para cada funcionalidad:

1. **Trabajas en la rama** (modificar archivos, hacer commits y push).
2. **Creas un Pull Request en GitHub** para fusionarlo en develop.
3. **Fusionas en develop y eliminas la rama local y remota.**

```
git branch -d feature/nombre-funcionalidad
```

```
git push origin --delete feature/nombre-funcionalidad
```

4. Lanzamiento de Versión Estable

Cuando todas las funcionalidades estén listas y probadas, se fusiona develop en main para lanzar la versión.

```
git checkout main
```

```
git merge develop
```

```
git push origin main
```

✓ Ahora main tiene la nueva versión de *Inventiory*.

5. Corrección de Errores en Producción (hotfix/)

Si hay un error en main, se crea una rama hotfix/ desde main para corregirlo rápidamente.

```
git checkout main
```

```
git checkout -b hotfix/correccion-login
```

```
git push origin hotfix/correccion-login
```

✦ Subir cambios y fusionarlo en main y develop:

```
git add .
```

```
git commit -m "Corrección de error en login"
```

```
git push origin hotfix/correccion-login
```

```
git checkout main
```

```
git merge hotfix/correccion-login
```

git push origin main

git checkout develop

git merge hotfix/correccion-login

git push origin develop

git branch -d hotfix/correccion-login

git push origin --delete hotfix/correccion-login

✓ El error está corregido en producción y en la versión en desarrollo.

6. Resumen Visual del Flujo en Git

main

└— develop

| └— feature/autenticacion-jwt

| └— feature/gestion-usuarios

| └— feature/gestion-clientes

| └— feature/gestion-productos

| └— feature/control-stock

| └— feature/registro-ventas

| └— feature/facturacion

| └— feature/informes

| └— hotfix/correccion-login

7. Conclusión

✓ Cada funcionalidad se desarrolla en una rama feature/.

✓ Se hacen commits y push regularmente.

✓ Se usa **Pull Request** para fusionar en develop.

- ✓ Cuando todo está listo, develop se fusiona en main.
- ✓ Los errores en producción se corrigen con hotfix/.